Complier design

1) i) construct LR(0) items, state transition diagram, Parsing table and perform Parsing on the input string $abab

ii) construct LR(1) items, state transition diagram, Parsing table. and perform Parsing on the input string $abab

use the grammar

$S \rightarrow AA$

$A \rightarrow aA | b$

=> Given grammar:

$S \rightarrow AA$

$A \rightarrow aA | b$

Input string: abab

Rule	Production
I_0	$S' \rightarrow S$
I_1	$S \rightarrow AA$
I_2	$A \rightarrow aA$
I_3	$A \rightarrow b$

$S' \to .S$
$S \to .AA$
$A \to .aA$
$A \to .b$
} I_0

$I_1 \to$ GOTO (I_0, S)
$S' \to S \to I_2$

$I_2 \to$ GOTO (I_0, A)
$S \to A.A$
$S \to .aA$
$A \to .b$

$I_3 \to$ GOTO (I_0, a)
$A \to a.A$
$A \to .aA$
$A \to .b$

$I_4 \to$ GOTO (I_0, b)
$A \to b.$

$I_5 \to$ GOTO (I_2, A)
$S \to AA.$

$I_6 \to$ GOTO (I_3, A)
$A \to aA.$

$S' \to .S$
$S \to .AA$
$A \to .aA$
$A \to .b$
} I_0

$I_1 \to$ GOTO (I_0, S)
$S' \to S \to I_1$

$I_2 \to$ GOTO (I_0, A)
$S \to A.A$
$S \to .aA$
$A \to .b$

$I_3 \to$ GOTO (I_0, a)
$A \to a.A$
$A \to .aA$
$A \to .b$

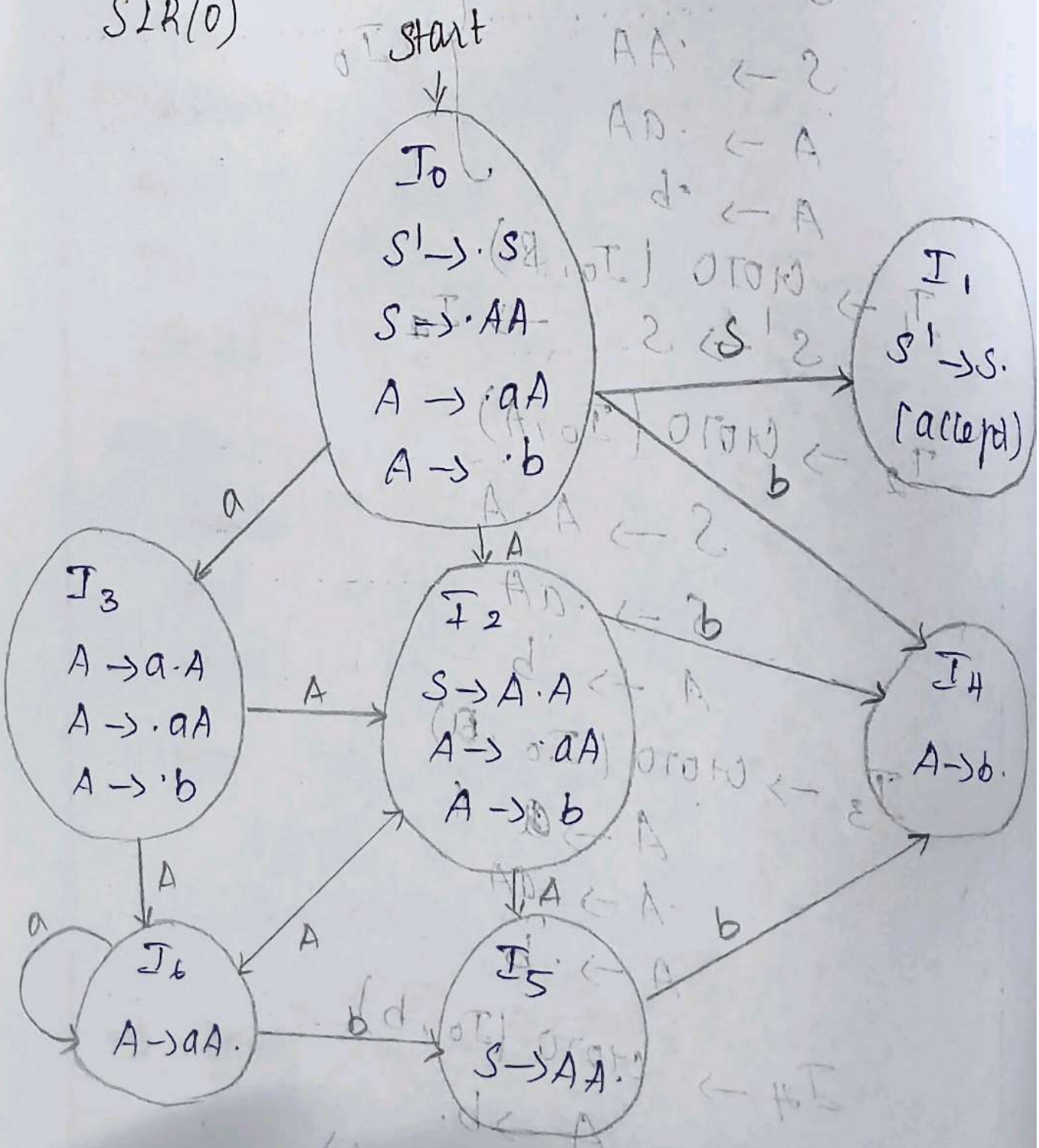
$I_4 \to$ GOTO (I_0, b)
$A \to b.$

$I_5 \to$ GOTO (I_2, A)
$S \to AA.$

$I_6 \to$ GOTO (I_3, A)

STATE TRANSITION

SLR(0)



Calculate FOLLOW sets

FOLLOW (A) = {a, b, $}

Parsing table:

State	Act(a)	Act(b)	Act($)	GOTO(S)	GOTO(A)
0	S_3	S_4	–	1	2
1	–	–	acc	–	–
2	S_3	S_4	–	–	5
3	S_3	S_4	–	–	6
4	r_3	r_3	r_3	–	–
5	–	–	r_1	–	–
6	r_2	r_2	r_2	–	–

Parse the Input string abab$

stack	Input	Action
0	abab$	shift 3
0 a 3	bab$	shift 4
0a 3b4	ab$	Reduce A→b
0a 3A 6	ab$	Reduce A→aA
0A 2	ab$	shift 3
0A2 a3	b$	shift 4
0A 2a3b4	$	Reduce A→b
0 A 2a 3A6	$	Reduce A→aA
0 A 2A 5	$	Reduce S→AA
		Accept

ii) CLR parsing using LR(1)

construct the LR(1)

$S' \rightarrow S$

$S \rightarrow AA$

$S \rightarrow aA \mid b$

*) $[S' \rightarrow \cdot S, \$]$
$[S \rightarrow \cdot AA, \$]$
$[A \rightarrow \cdot aA, a]$
$[A \rightarrow \cdot b, a]$
$[A \rightarrow \cdot aA, b]$
$[A \rightarrow \cdot b, b]$ } I_0

*) $I_1 \rightarrow [S' \rightarrow S\cdot, \$]$

*) $[S \rightarrow A \cdot A, \$]$
$[A \rightarrow \cdot aA, \$]$
$[A \rightarrow \cdot b, \$]$ } I_2

*) $[A \rightarrow a \cdot A, a]$
$[A \rightarrow a \cdot A, b]$ } I_3

*) I_4 → [A → b·, a]
[A → b·, b]

*) I_5 [S → A·A, $]

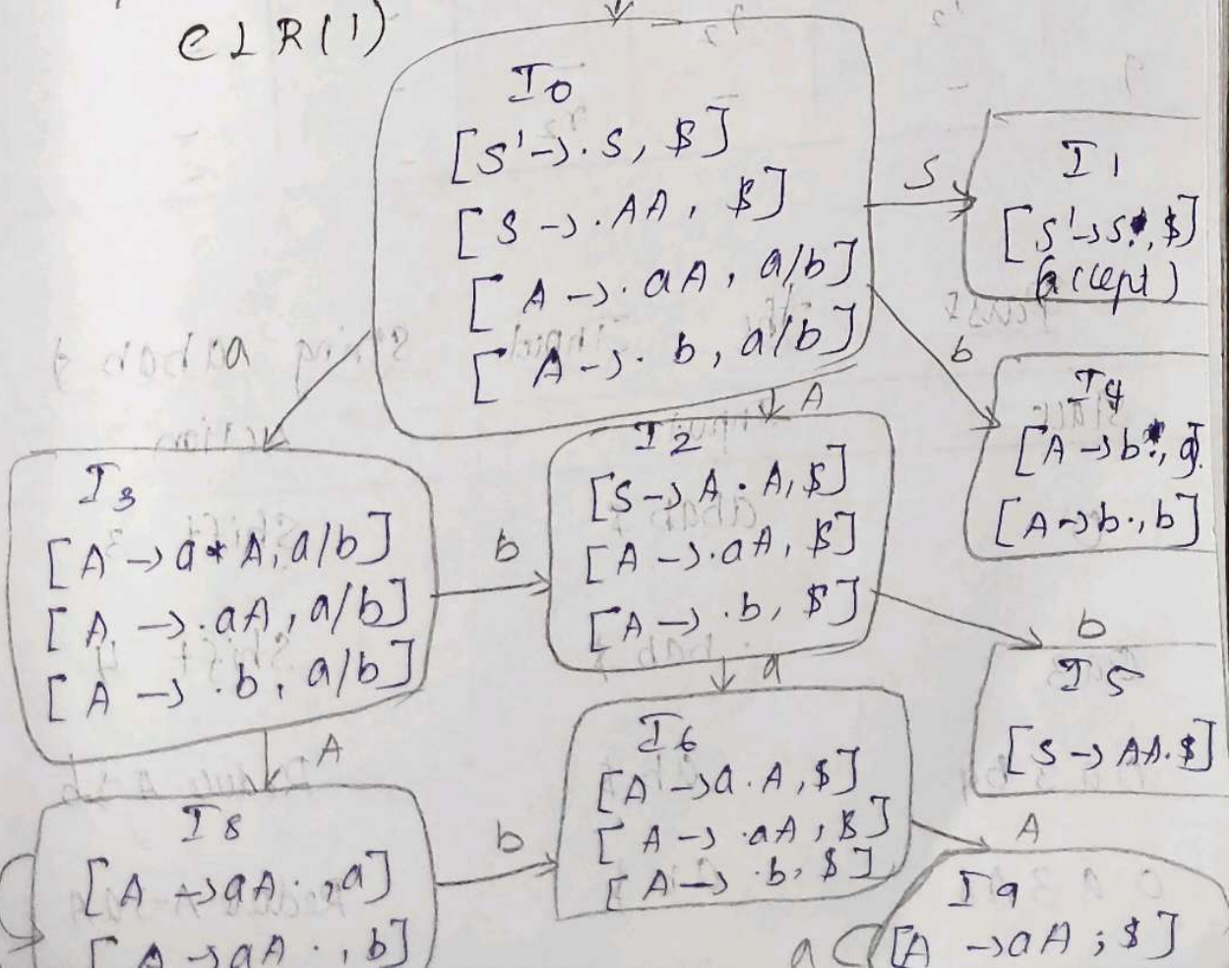
*) [A → a·A, $]
[A → ·aA, $]
[A → ·b, $]
} I_6

*) I_7 → [A → b·, $]

*) I_8 → [A → aA·, a]
[A → aA·, b]

*) I_9 → [A → aA·, $]

STATE DIAGRAM
CLR(1)



Construct Parsing table

State	Act(a)	Act(b)	Act(\$)	GOTO(S)	GOTO
0	s_3	s_4	–	1	2
1	–	–	acc	–	–
2	s_6	s_7	–	–	5
3	s_3	s_4	–	–	8
4	r_3	r_3	–		
5	–	–	r_1	–	–
6	s_6	s_7	–		–
7	–	–	r_3	–	9
8	r_2	r_2	–		
9	–	–	r_2	–	–

Parse the Input String abab$

Stack	Input	Action
0	abab$	Shift 3
0a3	bab$	Shift 4
0a3b4	ab$	Reduce A→b
0a3A8	ab$	Reduce A→aA

0A2	ab$	Shift 6
0A2a6	b$	Shift 7
0A2a6b7	$	Reduce A→b
0A2a6A9	$	Reduce A→aA
0A2A5	$	Reduce S→AA
0S1	$	Accept